

Fine-Tuning a Domain-Specific Academic Assistant (LLMs)

Course: COMP-341L (Artificial Neural Networks Lab)

Lab: 13

Date: May 04, 2026

Environment: Google Colab

Student: Ali Hamza

Roll No: B23F0063AI106

Section: B.S AI - Red

Method: LoRA (PEFT)

ABSTRACT

This lab develops and evaluates a domain-specific academic assistant for explaining Computer Science concepts in a consistent course-specific teaching style. A small causal language model (`distilgpt2`) is fine-tuned using LoRA adapters on a manually curated instruction-response dataset. We compare base vs fine-tuned outputs using a fixed evaluation prompt (“Explain dynamic programming”), analyze failure cases (hallucination, repetition, off-topic content), and perform one manual improvement iteration by refining the dataset.

Note: All dataset explanations were written manually. Model outputs are presented as raw evidence, followed by human analysis and reasoning.

KEYWORDS

LLM fine-tuning, LoRA, PEFT, `distilgpt2`, instruction tuning, hallucination, teaching style.

1. INTRODUCTION

General-purpose chatbots often respond with generic explanations, inconsistent tone, and may hallucinate details when a prompt is underspecified. In an academic setting, a course assistant must be both **clear** and **style-consistent**, and ideally reduce overconfident guessing. This lab targets style control via small-scale instruction tuning.

2. TEACHING STYLE DEFINITION (PART 2)

Strict teaching style template (enforced in the dataset):

- **Step-by-step** explanation (numbered)
- **Analogy** (real-life)
- **Tiny example** (1–3 lines)
- **Common mistake** / misconception
- **Recap** (one-line summary)

The goal is to teach the model a **repeatable structure** that improves readability for beginners and makes answers predictable for grading and classroom use.

3. DATASET CREATION (PART 1)

A manual instruction dataset was constructed (JSONL format) by writing original explanations for core CS topics (data structures, algorithms, ML basics, OS/security concepts). Each sample follows the strict teaching style template.

Dataset	Path	Samples
v1	<code>data/dataset_cs_instructions_v1.jsonl</code>	26
v2	<code>data/dataset_cs_instructions_v2.jsonl</code>	5

Example dataset item (excerpt)

```
Instruction: Explain recursion using a real-life analogy for a beginner.
Response (structure):
1) ... (step-by-step)
Analogy: ...
Tiny example: ...
Common mistake: ...
Recap: ...
```

4. MODEL SELECTION (PART 3)

Selected model: `distilgpt2` (small causal LM). It is suitable for a classroom lab because it trains quickly on limited compute and makes fine-tuning behavior observable.

Advantages: small, fast iteration, coherent short-form responses.

Limitations: weak factual reliability, limited reasoning depth, prone to repetition and off-topic text when data is small or decoding is stochastic.

5. FINE-TUNING STRATEGY (PART 4)

Method: LoRA (Low-Rank Adaptation) via PEFT. Instead of updating all model parameters, LoRA learns low-rank adapter matrices for selected attention projections. This reduces GPU memory usage and makes training feasible in Colab.

Why LoRA here:

- Trains fewer parameters → faster and cheaper
- Less risk of catastrophic forgetting in the base model
- Adapters can be saved/loaded independently for comparison

6. EXPERIMENTAL SETUP

- **Prompt format:** `### Instruction: ... ### Response:`
- **Fine-tuning objective:** next-token prediction (causal LM) on formatted instruction-response text
- **Evaluation prompt (fixed):** “Explain dynamic programming in a step-by-step, analogy-based teaching style.”
- **Decoding:** sampling enabled (`temperature=0.7`, `top_p=0.9`)

7. EVALUATION (PART 5)

The same evaluation prompt is used for base vs fine-tuned models.

Outputs below are copied from: `eval_base.txt`, `eval_finetuned_v1.txt`, and `eval_finetuned_v2.txt`.

7.1 Base model output

```
Primary
style-tuning
### Instruction:
Explain dynamic programming in a step-by-step, analogy-based teaching style.
Manual
improvement
iteration
This is a great introduction to the following topics, (failure-are covered in this tutorial.
driven: This is not a complete tutorial.
examples)
### The lesson is a part of the course, so please read the full course description.
```

7.2 Fine-tuned output (v1)

Instruction:

Explain dynamic programming in a step-by-step, analogy-based teaching style.

Response:

This post is part of a series that focuses on the language of programming in a way that is not currently possible in the language of programming. It has been translated into English and Spanish.

The most recent version of this article was translated into Spanish.

There are three main problems with the language of programming in a way that is not currently possible in the language of programming.

First, it is not possible to use a language of programming in a way that is not currently possible in the language of programming. The language of programming is not suitable for this purpose.

Second, it is not possible to use a language of programming in a way that is not currently possible in the language of programming. The language of programming is not suitable for this purpose.

Third, it is not possible to use a language of programming in a way that is not currently possible in the language of programming. The language of programming is not suitable for this purpose.

The problem is that the language of programming is not suitable for this purpose.

The problem is that the language of programming is not suitable for this purpose.

7.3 Fine-tuned output (v2, after manual improvement)

```
### Instruction:
Explain dynamic programming in a step-by-step, analogy-
based teaching style.
```

```
### Response:
"I am going to say that I am going to say that I am
going to say that I am going to say that I am going to
say that I am going to say that I am going to say that
I am going to say that I am going to say that I am
going to say that I am going to say that I am going to
say that I am going to say that I am going to say that
I am going to say that I am going to say that I am
going to say that I am going to say that I am going to
say that I am going to say that I am going to say that
I am going to say that I am going to say that I am
going to say that I am going to say that I am going to
say that I am going to say that I am going to say that
I am going
```

7.4 Critical comparison (analysis)

Clarity: Base output is short but does not explain dynamic programming. Fine-tuned v1/v2 outputs are verbose but largely off-topic and repetitive, failing to follow the intended teaching structure.

Hallucination reduction: The tuned models did not become more cautious; instead they produced “article-like” filler text. This indicates the fine-tuning did not successfully imprint the desired format.

Style consistency: The strict template (steps + analogy + example + mistake + recap) is not consistently produced. This suggests either (i) the dataset is too small for stable behavioral change, (ii) training overfit to noise patterns, or (iii) decoding settings increased repetition.

8. FAILURE ANALYSIS (PART 6)

Two failure prompts were used to diagnose behavior. Raw outputs are stored in JSON files and summarized here.

Failure case 1: “Prove DP always gives optimal answer”

Observed failure: Both tuned models avoid an actual proof and instead generate irrelevant, repetitive filler.

Why it failed: The dataset teaches explanation style but does not include proof-like answers, formal assumptions (optimal substructure), or examples of when a “proof request” should be answered carefully.

Cause category: primarily **data limitation** (missing proof-oriented examples) plus **model limitation** (small model).

Failure case 2: “Exact best-known TSP complexity + year”

Observed failure: The base model replies with a vague update statement; tuned models generate confident-sounding content (including a suspicious arXiv link) without actually answering precisely.

Why it failed: The prompt requests precise external facts (algorithm details + discovery year). A small model may guess, and the fine-tuning set lacks examples that teach “I’m not sure / I don’t have a citation.”

Cause category: model limitation + training data does not include uncertainty handling.

9. MANUAL IMPROVEMENT (PART 7)

A manual improvement iteration was performed by creating `dataset_cs_instructions_v2.jsonl`, focusing on:

- Explicit “how to choose a DP state” examples (reduces vague DP answers)
- Examples explaining why small LLMs hallucinate and how to respond cautiously

Result: In the captured v2 evaluation output, repetition increased rather than decreasing. This suggests that the fix was insufficient because the improvement set is extremely small (5 items) and cannot counteract instability from stochastic decoding.

Recommendations (what to try next)

- Increase dataset size (e.g., 80–200 high-quality style-consistent samples) with more DP-focused items.
- Use deterministic evaluation (greedy decoding) for fair comparisons; keep sampling only for creativity tests.
- Add “refusal / uncertainty” examples to discourage confident guessing when asked for citations/years.
- Lower learning rate / epochs to reduce repetition; add early stopping on training loss trends.
- Use a slightly stronger small model (e.g., GPT-2 medium) if compute allows.

10. CONCLUSION

This lab demonstrates that fine-tuning a small LLM with very limited data can fail to reliably enforce a desired teaching style. The most important practical insight is that **dataset quality and quantity dominate**: to achieve consistent behavior, we need more diverse, high-signal examples aligned with the target tasks and evaluation prompts.

REFERENCES

1. Hugging Face, “Transformers” (library documentation).
2. PEFT (Parameter-Efficient Fine-Tuning) library documentation.
3. DistilGPT-2 / GPT-2 model cards and related documentation.
4. LoRA: Low-Rank Adaptation technique for parameter-efficient fine-tuning.